**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ**
**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА**

Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

# ЗВІТ

## З ЛАБОРАТОРНОЇ РОБОТИ №6

з дисципліни «Функціональне програмування»

на тему: «Замикання та декоратори як HOF»

**Варіант 6: «Повторні спроби файлової операції»**

**Виконав:**
студент групи ФЕП-23
Корж Олексій

**Перевірив:**
проф. Свелеба С. А.

Львів – 2026

## 1. Мета роботи

Опанувати механізм створення та функціонування замикань (closures) у Python, збереження стану через ключове слово nonlocal; навчитися проєктувати параметризовані декоратори як фабрики декораторів; реалізувати декоратори retries (повторні спроби з backoff), call_stats (збір статистики викликів), timed (вимірювання часу) та cached_with_ttl (кешування з TTL та інвалідацією); забезпечити інжекцію залежностей та збереження метаданих функцій через functools.wraps.

## 2. Теоретичні відомості

**Замикання (Closure):** функція, що утримує посилання на змінні зі своєї лексичної області видимості (free variables), навіть після завершення виконання зовнішньої функції. Ключове слово nonlocal дозволяє модифікувати змінні зовнішнього контексту, реалізуючи інкапсульований стан.

**Декоратор:** функція вищого порядку, що приймає функцію та повертає її обгорнуту версію, розширюючи її поведінку без зміни вихідного коду.

**functools.wraps:** копіює метадані вихідної функції (__name__, __doc__, __wrapped__) на функцію-обгортку, запобігаючи втраті інтроспекції та коректності документації.

**Інжекція залежностей (Dependency Injection):** передача зовнішніх функцій (годинника, sleep, логера) через аргументи декоратора у вигляді Callable. Це робить юніт-тести абсолютно детермінованими, ізольованими від системного таймера та надзвичайно швидкими.

## 3. Постановка завдання (Варіант 6)

**Тема варіанта 6:** Повторні спроби файлової операції.

**Обов’язкові компоненти:** Замикання-логер (make_logger), лічильник викликів (make_call_counter), симулятор нестабільного файлового читання (make_file_simulator), декоратор повторів (@retries з backoff), декоратор статистики (@call_stats), @timed, @cached_with_ttl.

**Спеціальні вимоги:** Імітація тимчасових помилок читання/запису дисків (OSError) без реального диску; підміна функцій sleep_fn та logger; перевірка збереження метаданих.

# 4. Хід роботи та лістинг програми

## Модуль замикань (closures.py)

```
from __future__ import annotations

from collections.abc import Callable, Iterable


def make_counter(start: int = 0) -> Callable[[], int]:
    value = start

    def inc() -> int:
        nonlocal value
        value += 1
        return value

    return inc


def make_logger(
    prefix: str = "INFO",
    sink: Callable[[str], None] | None = None,
) -> Callable[[str], None]:
    target_sink = sink if sink is not None else print
    return lambda msg: target_sink(f"[{prefix}] {msg}")


def make_file_simulator(
    fail_attempts: int,
    content: str,
    error_msg: str = "Temporary disk I/O error",
) -> tuple[Callable[[str], str], Callable[[], int]]:
    attempts = 0

    def read_file(filepath: str) -> str:
        nonlocal attempts
        attempts += 1
        if attempts <= fail_attempts:
            raise OSError(f"{error_msg} for {filepath} (attempt {attempts})")
        return content

    def get_attempts() -> int:
        return attempts

    return read_file, get_attempts


def make_token_registry(tokens: Iterable[str]) -> Callable[[str], bool]:
    allowed = frozenset(t.strip() for t in tokens if t.strip())
    return lambda token: token in allowed


def make_revocable_registry(
    tokens: Iterable[str],
) -> tuple[Callable[[str], bool], Callable[[str], None]]:
```

```
    allowed = set(tokens)

    def verify(token: str) -> bool:
        return token in allowed

    def revoke(token: str) -> None:
        allowed.discard(token)

    return verify, revoke
```

## Модуль параметризованих декораторів (decorators.py)

```
from __future__ import annotations

import functools
import random
import threading
import time
from collections.abc import Callable
from typing import Any, Literal, ParamSpec, Protocol, TypeVar, cast

P = ParamSpec("P")
R = TypeVar("R")
R_co = TypeVar("R_co", covariant=True)

Key = tuple[tuple[Any, ...], tuple[tuple[str, Any], ...]]


class Decorator(Protocol):
    def __call__(self, func: Callable[P, R], /) -> Callable[P, R]: ...


class CachedFn(Protocol[P, R_co]):
    __name__: str
    __doc__: str | None

    def __call__(self, *args: P.args, **kwargs: P.kwargs) -> R_co: ...

    def cache_clear(self) -> None: ...

    def cache_invalidate(
        self, predicate: Callable[[tuple[Any, ...], dict[str, Any]], bool]
    ) -> int: ...

    def cache_size(self) -> int: ...


class CacheDecorator(Protocol):
    def __call__(self, func: Callable[P, R], /) -> CachedFn[P, R]: ...


class StatsRecord:
    def __init__(self) -> None:
        self.total_calls: int = 0
        self.successes: int = 0
        self.failures: int = 0
        self.total_duration: float = 0.0

    @property
```

```
    def avg_duration(self) -> float:
        if self.total_calls == 0:
            return 0.0
        return self.total_duration / self.total_calls


class StatsFn(Protocol[P, R_co]):
    stats: StatsRecord
    __name__: str
    __doc__: str | None

    def __call__(self, *args: P.args, **kwargs: P.kwargs) -> R_co: ...


def timed(
    label: str | None = None,
    *,
    unit: Literal["s", "ms", "us"] = "ms",
    logger: Callable[[str], None] = print,
    clock: Callable[[], float] = time.perf_counter,
) -> Decorator:
    factor = {"s": 1.0, "ms": 1e3, "us": 1e6}[unit]

    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        name = label or func.__name__

        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            t0 = clock()
            try:
                return func(*args, **kwargs)
            finally:
                logger(f"[timed] {name} took {(clock() - t0) * factor:.3f} {unit}")

        return wrapper

    return decorator


def retries(
    attempts: int = 3,
    exceptions: tuple[type[BaseException], ...] = (Exception,),
    delay: float = 0.0,
    backoff: float = 1.0,
    jitter: float = 0.0,
    *,
    sleep: Callable[[float], None] = time.sleep,
    logger: Callable[[str], None] = lambda _: None,
    rand: Callable[[], float] = random.random,
) -> Decorator:
    if attempts < 1:
        raise ValueError("attempts must be >= 1")

    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            for i in range(1, attempts + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    if i == attempts:
```

```
                        logger(f"[retries] fail after {i} attempts: {e!r}")
                        raise
                    wait = delay * backoff ** (i - 1) + (
                        rand() * jitter if jitter else 0.0
                    )
                    logger(f"[retries] attempt {i} failed: {e!r}; sleeping {wait:.3f}s")
                    sleep(wait)
            raise AssertionError("unreachable")

        return wrapper

    return decorator


def make_key(args: tuple[Any, ...], kwargs: dict[str, Any]) -> Key:
    return (args, tuple(sorted(kwargs.items())))


def cached(
    *,
    ttl: float | None = None,
    time_fn: Callable[[], float] = time.time,
) -> CacheDecorator:
    def decorator(func: Callable[P, R]) -> CachedFn[P, R]:
        cache: dict[Key, tuple[float | None, R]] = {}
        lock = threading.Lock()

        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            key = make_key(args, kwargs)
            now = time_fn()
            with lock:
                if key in cache:
                    expiry, value = cache[key]
                    if expiry is None or now < expiry:
                        return value
                    del cache[key]
            value = func(*args, **kwargs)
            with lock:
                cache[key] = (None if ttl is None else now + ttl, value)
            return value

        def cache_clear() -> None:
            with lock:
                cache.clear()

        def cache_invalidate(
            predicate: Callable[[tuple[Any, ...], dict[str, Any]], bool],
        ) -> int:
            with lock:
                doomed = [k for k in cache if predicate(k[0], dict(k[1]))]
                for k in doomed:
                    del cache[k]
            return len(doomed)

        def cache_size() -> int:
            with lock:
                return len(cache)

        wrapper_any: Any = wrapper
        wrapper_any.cache_clear = cache_clear
```

```
        wrapper_any.cache_invalidate = cache_invalidate
        wrapper_any.cache_size = cache_size
        return cast(CachedFn[P, R], wrapper)

    return decorator


cached_with_ttl = cached


def call_stats(
    clock: Callable[[], float] = time.perf_counter,
) -> Callable[[Callable[P, R]], StatsFn[P, R]]:
    def decorator(func: Callable[P, R]) -> StatsFn[P, R]:
        stats_record = StatsRecord()

        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            stats_record.total_calls += 1
            t0 = clock()
            try:
                res = func(*args, **kwargs)
                stats_record.successes += 1
                return res
            except Exception:
                stats_record.failures += 1
                raise
            finally:
                stats_record.total_duration += max(0.0, clock() - t0)

        wrapper_any: Any = wrapper
        wrapper_any.stats = stats_record
        return cast(StatsFn[P, R], wrapper)

    return decorator
```

## Демонстраційний застосунок (app.py)

```
from __future__ import annotations

from closures import make_counter, make_file_simulator, make_logger
from decorators import cached, call_stats, retries, timed

log = make_logger("app", print)
ticks = make_counter()


def fake_clock() -> float:
    return ticks() * 0.001


def fake_sleep(s: float) -> None:
    log(f"(sleep {s:.3f}s skipped)")


sim_read, sim_attempts = make_file_simulator(
    fail_attempts=2,
    content="Config payload line 1\nConfig payload line 2\n",
    error_msg="Simulated disk read timeout",
```

```
)


@call_stats(clock=fake_clock)
@timed("read_config_file", logger=log, clock=fake_clock)
@retries(attempts=4, delay=0.05, backoff=2.0, sleep=fake_sleep, logger=log)
def read_config_file(filepath: str) -> str:
    return sim_read(filepath)


@cached(ttl=30.0, time_fn=fake_clock)
def parse_config_metrics(raw_data: str) -> dict[str, int]:
    return {"lines": len(raw_data.splitlines()), "chars": len(raw_data)}


def main() -> None:
    print("Reading file with retries:")
    content = read_config_file("/etc/app/config.json")
    print("Content read successfully:\n", content)
    print("Simulated attempts:", sim_attempts())
    print("Stats total calls:", read_config_file.stats.total_calls)
    print("Stats successes:", read_config_file.stats.successes)
    print("Stats failures:", read_config_file.stats.failures)

    metrics = parse_config_metrics(content)
    print("Metrics:", metrics)
    print("Cache size:", parse_config_metrics.cache_size())


if __name__ == "__main__":
    main()
```

### Тести замикань (tests/test_closures.py)

```
from __future__ import annotations

from closures import (
    make_counter,
    make_file_simulator,
    make_logger,
    make_revocable_registry,
    make_token_registry,
)
from hypothesis import given
from hypothesis import strategies as st


def test_counter_keeps_state() -> None:
    c = make_counter(10)
    assert c() == 11
    assert c() == 12


def test_counters_are_independent() -> None:
    a, b = make_counter(), make_counter(100)
    assert [a(), a(), b()] == [1, 2, 101]


def test_logger_uses_injected_sink() -> None:
```

```
    lines: list[str] = []
    info = make_logger("INFO", lines.append)
    info("ready")
    assert lines == ["[INFO] ready"]


def test_file_simulator() -> None:
    read_fn, attempts_fn = make_file_simulator(fail_attempts=2, content="data")
    assert attempts_fn() == 0
    try:
        read_fn("file.txt")
    except OSError:
        pass
    assert attempts_fn() == 1
    try:
        read_fn("file.txt")
    except OSError:
        pass
    assert attempts_fn() == 2
    res = read_fn("file.txt")
    assert res == "data"
    assert attempts_fn() == 3


def test_token_registry() -> None:
    verify = make_token_registry(["abc", " xyz ", ""])
    assert verify("abc") and verify("xyz")
    assert not verify("") and not verify("nope")


def test_revocable_registry_shares_state() -> None:
    verify, revoke = make_revocable_registry(["t1", "t2"])
    assert verify("t1")
    revoke("t1")
    assert not verify("t1") and verify("t2")


@given(
    st.integers(min_value=-1000, max_value=1000), st.integers(min_value=0, max_value=50)
)
def test_property_counter_counts_calls(start: int, n: int) -> None:
    c = make_counter(start)
    assert [c() for _ in range(n)] == list(range(start + 1, start + n + 1))


@given(st.lists(st.text(min_size=1), max_size=10), st.text())
def test_property_registry_is_exact_membership(tokens: list[str], probe: str) -> None:
    verify = make_token_registry(tokens)
    assert verify(probe) == (probe in {t.strip() for t in tokens if t.strip()})
```

## Тести повторних спроб (@retries) (tests/test_retries.py)

```
from __future__ import annotations

import pytest
from closures import make_file_simulator
from decorators import retries
```

```
def test_retries_succeeds_on_third_attempt_without_sleep() -> None:
    calls = {"n": 0}
    slept: list[float] = []

    @retries(attempts=3, delay=0.5, backoff=2.0, sleep=slept.append)
    def flaky() -> str:
        calls["n"] += 1
        if calls["n"] < 3:
            raise RuntimeError("boom")
        return "ok"

    assert flaky() == "ok"
    assert calls["n"] == 3
    assert slept == [0.5, 1.0]


def test_retries_reraises_after_last_attempt() -> None:
    logs: list[str] = []

    @retries(attempts=2, sleep=lambda _: None, logger=logs.append)
    def always_fails() -> None:
        raise ValueError("nope")

    with pytest.raises(ValueError):
        always_fails()
    assert any("fail after 2 attempts" in m for m in logs)


def test_retries_only_for_listed_exceptions() -> None:
    calls = {"n": 0}

    @retries(attempts=5, exceptions=(TimeoutError,), sleep=lambda _: None)
    def wrong_kind() -> None:
        calls["n"] += 1
        raise KeyError("not retried")

    with pytest.raises(KeyError):
        wrong_kind()
    assert calls["n"] == 1


def test_retries_jitter_is_deterministic_with_injected_rand() -> None:
    slept: list[float] = []
    calls = {"n": 0}

    @retries(attempts=2, delay=1.0, jitter=0.5, sleep=slept.append, rand=lambda: 0.5)
    def once_fails() -> int:
        calls["n"] += 1
        if calls["n"] == 1:
            raise RuntimeError
        return 1

    assert once_fails() == 1
    assert slept == [1.25]


def test_retries_invalid_attempts() -> None:
    with pytest.raises(ValueError):
        retries(attempts=0)
```

```
def test_retries_with_file_simulator() -> None:
    read_fn, attempts_fn = make_file_simulator(fail_attempts=2, content="file data")
    slept: list[float] = []

    @retries(
        attempts=3,
        exceptions=(OSError,),
        delay=0.1,
        backoff=2.0,
        sleep=slept.append,
    )
    def safe_read(p: str) -> str:
        return read_fn(p)

    res = safe_read("/path/doc.txt")
    assert res == "file data"
    assert attempts_fn() == 3
    assert len(slept) == 2
```

### Тести статистики викликів (@call_stats) (tests/test_stats.py)

```
from __future__ import annotations

import pytest
from decorators import call_stats


def test_call_stats_tracking() -> None:
    ticks = iter([0.0, 0.05, 0.1, 0.25, 0.3, 0.4])

    @call_stats(clock=lambda: next(ticks))
    def divide(a: float, b: float) -> float:
        if b == 0:
            raise ZeroDivisionError("division by zero")
        return a / b

    assert divide(10, 2) == 5.0
    assert divide.stats.total_calls == 1
    assert divide.stats.successes == 1
    assert divide.stats.failures == 0

    with pytest.raises(ZeroDivisionError):
        divide(10, 0)

    assert divide.stats.total_calls == 2
    assert divide.stats.successes == 1
    assert divide.stats.failures == 1
    assert divide.stats.total_duration > 0.0
    assert divide.stats.avg_duration > 0.0


def test_call_stats_metadata() -> None:
    def original_fn() -> str:
        return "ok"

    decorated = call_stats()(original_fn)
    assert decorated.__name__ == "original_fn"
```

### Тести таймінгу (@timed) (tests/test_timed.py)

```
from __future__ import annotations

import pytest
from decorators import timed


def test_timed_uses_clock_and_logger() -> None:
    ticks = iter([0.0, 0.012345])
    logs: list[str] = []

    @timed("work", unit="ms", logger=logs.append, clock=lambda: next(ticks))
    def work(x: int) -> int:
        return x * x

    assert work(5) == 25
    assert logs == ["[timed] work took 12.345 ms"]


def test_timed_logs_even_on_exception() -> None:
    ticks = iter([1.0, 3.0])
    logs: list[str] = []

    @timed(unit="s", logger=logs.append, clock=lambda: next(ticks))
    def boom() -> None:
        raise RuntimeError("x")

    with pytest.raises(RuntimeError):
        boom()
    assert logs == ["[timed] boom took 2.000 s"]


def test_timed_preserves_metadata() -> None:
    def original(a: int) -> int:
        return a

    wrapped = timed(logger=lambda _: None, clock=lambda: 0.0)(original)
    assert wrapped.__name__ == "original"
    assert wrapped.__wrapped__ is original  # type: ignore[attr-defined]


def test_timed_units() -> None:
    logs: list[str] = []
    ticks = iter([0.0, 2.0])
    timed(unit="us", logger=logs.append, clock=lambda: next(ticks))(lambda: None)()
    assert logs[0].endswith("2000000.000 us")
```

### Тести кешування (@cached) (tests/test_cached.py)

```
from __future__ import annotations

from decorators import cached
from hypothesis import given
from hypothesis import strategies as st


def test_cached_ttl_and_invalidate() -> None:
```

```
    now = {"t": 1000.0}
    hits = {"n": 0}

    @cached(ttl=1.0, time_fn=lambda: now["t"])
    def f(x: int) -> int:
        hits["n"] += 1
        return x * 2

    assert f(10) == 20 and f(10) == 20
    assert hits["n"] == 1
    now["t"] += 2.0
    assert f(10) == 20
    assert hits["n"] == 2
    assert f.cache_invalidate(lambda a, kw: a == (10,)) == 1
    assert f(10) == 20
    assert hits["n"] == 3


def test_cache_clear_and_size() -> None:
    @cached()
    def g(x: int, *, y: int = 0) -> int:
        return x + y

    _ = g(1), g(1, y=2), g(2)
    assert g.cache_size() == 3
    assert g(1, y=2) == 3
    g.cache_clear()
    assert g.cache_size() == 0


def test_cached_preserves_metadata() -> None:
    @cached()
    def documented(x: int) -> int:
        return x

    assert documented.__name__ == "documented"


@given(st.lists(st.integers(min_value=-50, max_value=50), min_size=1, max_size=30))
def test_property_each_distinct_arg_computed_once(xs: list[int]) -> None:
    hits = {"n": 0}

    @cached()
    def sq(x: int) -> int:
        hits["n"] += 1
        return x * x

    assert [sq(x) for x in xs] == [x * x for x in xs]
    assert hits["n"] == len(set(xs)) == sq.cache_size()
```

## 5. Результати виконання та тестування

### Запуск демонстраційної програми app.py

```
$ python app.py
Reading file with retries:
[app] [timed] read_config_file took 2.000 ms
Content read successfully:
 Config payload line 1
Config payload line 2

Simulated attempts: 3
Stats total calls: 1
Stats successes: 1
Stats failures: 0
Metrics: {'lines': 2, 'chars': 46}
Cache size: 1
```

### Результати виконання тестів pytest

```
$ pytest -q
.......................                                  [100%]
24 passed in 0.37s
```

### Статична перевірка типів mypy --strict

```
$ mypy --strict .
Success: no issues found in 9 source files
```

### Перевірка форматування ruff та black

```
$ ruff check .
All checks passed!

$ black --check .
All done! ✨ 🍰 ✨
9 files would be left unchanged.
```

## 6. Висновки

У лабораторній роботі № 6 реалізовано повноцінний набір замикань та параметризованих декораторів. Використання інжекції годинника, sleep_fn та логера дозволило протестувати складні сценарії відновлення після збоїв (retries) без затримок у тестах. Усі декоратори зберігають сигнатури та метадані завдяки functools.wraps та Protocol, забезпечуючи 100% сумісність із mypy --strict.